Junior Developer Code Prompt Pack

Learn and build faster with 60+ curated coding prompts that teach.

For junior developers, bootcamp graduates, and self-taught programmers.

You learned to build a web app. Now you need to debug a crash you've never seen, review a pull request, explain your code in an interview, or understand a codebase that was written by six people over four years. Real junior devs hit these walls every day — and most of the time, speed of typing isn't the problem. The problem is *not knowing what to ask.*

This pack is a library of 60+ copy-pasteable prompt templates built to do one thing: **make AI a tutor, not a substitute.** Every prompt forces the AI to explain its reasoning, show alternatives, and ask you to verify before it writes a line of code. You stay in control of every change that ships. You learn the *why*, not just the *what*.

Every prompt uses `{{placeholders}}` you fill in with your actual code, error, or stack trace. Each one is paired with "when to use it" guidance and a description of what a good output looks like — so you know whether the answer you got is worth acting on.

How to use this pack

1. **Grab the prompt, fill in the blanks.** Copy the fenced block, replace every `{{placeholder}}` with your real code, error message, or goal. If a placeholder asks for your file and you can't paste it all, paste the relevant function or class plus a one-line description of the rest.
2. **Keep your code in the conversation.** Don't ask an AI to "fix my login." Paste the actual function and the actual error. The prompt templates are written for that, and they will push back if you give them too little.
3. **Use the "force explanation" technique.** Many prompts end with a request for the AI to explain *why* in terms you can verify. If the AI just dumps a rewritten file without explaining, respond with: *"Explain the reasoning behind each change, and tell me what to test to verify it."*
4. **Treat output as a draft, not a verdict.** The prompts are built to keep you in control. Read every suggested change. If you don't understand a line, ask the follow-up prompt: *"Explain line {{X}} as if I'm a junior developer."*
5. **Work through the 30-day plan.** The pack is most powerful used on a schedule — see **Your first 30 days** at the end. Don't try to use all 60 prompts in a week; pick the ones that match what you're actually doing.

The one rule of this pack: you ship the code, not the AI. If a prompt would let the AI silently rewrite your whole file and call it done, you're using the prompt wrong. Every template here is designed to stop that.

Explain Like I'm a Junior (ELI5J) — 10 patterns

These ten patterns are the foundation of the pack. They force the AI to teach at your level instead of flexing. Each one is a prompt template you can drop any code, concept, or error into. Master these and the rest of the pack gets easier.

Pattern 1: The Concept Explainer

Use when: you've hit a term in docs, a blog post, or a code review that you sort-of know but can't explain out loud.

```
Explain {{concept}} to me as if I'm a junior developer with {{X}} months of experience.

I need:
1. A plain-language definition (max 3 sentences) with a real-world analogy.
2. When this concept is actually used in a real codebase — give me 2-3 concrete examples.
3. A minimal code example in {{language}} that shows it in action.
4. The 2 most common mistakes juniors make with it, and how to avoid them.
5. One question I can ask myself to check I understand it.

Don't just define it. Teach it. If you'd need more than 300 words, ask me which part to go deeper on.
```

Worked example — the prompt filled in:

> Explain `async/await` to me as if I'm a junior developer with 6 months of experience. I need: 1. A plain-language definition (max 3 sentences) with a real-world analogy. 2. When this is actually used in a real codebase — give me 2-3 concrete examples. 3. A minimal code example in TypeScript that shows it in action. 4. The 2 most common mistakes juniors make with it, and how to avoid them. 5. One question I can ask myself to check I understand it. Don't just define it. Teach it. If you'd need more than 300 words, ask me which part to go deeper on.

What a good junior-level answer looks like (shortened):

> **Definition:** `async/await` is syntax that lets you write code that *waits* for a slow operation (like a network request or a database read) without freezing your whole program. Analogy: you order coffee at a café — you don't stand at the counter blocking everyone behind you; you get a number and pick it up when it's ready. `await` is "order a coffee and text me when it's done."
>
> **Where it's used:** (1) fetching data from an API in a React component, (2) reading from a database in a Node/Express route, (3) saving a file in a desktop app.
>
> **Minimal example:**
> `ts async function getUser(id: string): Promise<User> { const res = await fetch(`/api/users/${id}`); // wait for the network if (!res.ok) throw new Error(`Request failed: ${res.status}`); return res.json(); // this is NOT awaited — it returns a promise… // … actually it IS fine here because return unwraps it. }`
>
> **Two common mistakes:** (1) forgetting `await` — the code moves on before the promise resolves, so you get a `Promise` object instead of the data; (2) using `await` inside a `.map()` in a loop, which makes requests run one after another instead of in parallel — use `Promise.all` there.
>
> **Self-check question:** "Can I explain, without looking, what a promise is and what states it can be in?" If not, that's your next study topic.

Pattern 2: Error Decoder

Use when: you have a stack trace or error message and no idea what it means.

```
Here is an error I got in {{language}} / {{framework}}:

{{paste the full error message and stack trace}}

The code that produced it (the relevant function only):

{{paste code}}

Please:
1. Translate the error into plain English (one sentence, no jargon).
2. Point to the EXACT line in my code that most likely caused it, with an explanation of why.
3. Give me the 2-3 most likely causes, ordered by probability.
4. Suggest the smallest next step to confirm which cause it is (a log, a console.log, or a test).
5. Do NOT rewrite the whole file. Show me only the minimal fix for the most likely cause.

If you're not sure, say 'I'm not sure' and tell me what to check to find out.
```

Worked example:

> *(User pastes a TypeError: Cannot read property 'map' of undefined and a component —)*
>
> Here is an error I got in TypeScript / React: `TypeError: Cannot read properties of undefined (reading 'map')`. The code: `export default function Cart({ items }) { return <ul>{items.map(i => <li>{i.name}</li>)}</ul> }`
>
> **Plain English:** You're trying to loop over a list, but `items` is `undefined` — there's no list there.
>
> **Likely cause:** your component is being rendered before the data arrives (e.g. the parent fetches it asynchronously), so `items` is `undefined` on the first render.
>
> **Minimal fix:** give `items` a default: `export default function Cart({ items = [] }) {...}` — or conditional-render when the fetch is still loading. Don't rewrite the file; that one change fixes the crash.
>
> **Verify:** add `console.log('items', items)` before the return to see that the first render gets `undefined`, then confirm the fix silences it.

Pattern 3: "Review My Code" (teaching mode)

Use when: you want honest feedback on code you wrote — not a rewrite.

```
Review this code I wrote as if I'm a junior developer. I want to LEARN, not just get a fix.

{{paste your code — one file or one function}}

Please give me:
1. Three things I did well (be specific — quote the lines).
2. The top 3 problems, ordered by importance. For each: what's wrong, why it matters, and how to
fix it.
3. One suggestion to make the code more readable.
4. One suggestion to make it more testable.
5. One "if I was more senior I'd know this" tip.

Do NOT provide a full rewrite. Point me at the problems and let me fix them myself. If there's a
bug, tell me where to look, not the exact line to change... unless the bug is subtle enough that
I'd waste an hour finding it.
```

What a good output looks like: specific line references, a "well done" list that isn't generic, problems framed as questions ("what happens if `user` is null?") rather than commands, and exactly one stretch tip — not a firehose of senior-level advice.

Pattern 4: Code Explainer

Use when: you're reading unfamiliar code — someone else's PR, a library's source, code you wrote three months ago.

```
Explain this code to me as if I'm a junior developer. Read it line by line.

{{paste the code}}

I need:
1. A one-paragraph overview of what this block does and why it exists.
2. A line-by-line walkthrough, but ONLY for the lines that matter. Skip boilerplate like imports
and closing braces.
3. For each non-obvious line, tell me *why* it's written that way.
4. Point out anything weird, fragile, or that smells like a code smell, and explain why it's risky.
5. End with one question I should ask the person who wrote it.

If a line is doing something clever, tell me. If something is needlessly complicated, say so — I
want to know what good looks like.
```

Pattern 5: Model My Thinking

Use when: you're stuck on a design or algorithm problem — often for a take-home challenge or whiteboard interview.

```
I'm working on this problem and I'm stuck. Don't give me the answer yet.

Problem: {{describe the problem}}

What I've tried: {{describe your attempts and where you got stuck}}

Please:
1. Ask me 3 questions that would help me find the answer myself. Wait for my answers.
2. Then, help me break the problem into smaller steps.
3. Show me the approach (not the full code) in pseudocode or a high-level outline.
4. Point out the specific misconception that's probably blocking me, if you can see one.5. Let me
write the code. After I do, review it using Pattern 3.

Do NOT solve the whole thing for me. Act as a coach. If I ask you to just solve it, gently refuse
and ask me what I've tried.
```

Pattern 6: Compare The Options

Use when: you know two (or more) ways to do something and can't decide — or you're choosing between libraries.

```
I'm trying to choose between these options for {{goal}}. Help me decide.

Option A: {{describe}}
Option B: {{describe}}
(Add more if needed.)

Context: {{my project, team size, experience level, and constraints}}

Please give me:
1. A table comparing them on: learning curve, maintenance burden, performance, community/
popularity, and "how likely a senior dev is to approve it."
2. For the winner: a short explanation of *why* it wins for MY situation (not in general).
3. When the other option would actually be the better choice.
4. One tradeoff I'm probably not thinking about.

Don't just pick one. Teach me how to make this kind of call myself next time.
```

Pattern 7: The Perfect Question

Use when: you're about to ask for help in a team chat, Stack Overflow, or a forum — and you want to ask a question people will actually answer.

```
I'm about to ask a question about {{topic}} on {{where — Slack, Stack Overflow, etc.}}. Help me
write it so a senior developer will actually want to answer it.

Here's what I want to know: {{what you need}}

Here's what I've already tried: {{attempts}}

Please:
1. Write the question for me, in the tone and format of that platform.
2. Include the minimal code snippet that reproduces the problem.
3. Tell me what I'm probably missing that a senior dev would notice (the thing I should check
before posting).
4. Give me a version of the question with a clear "what I expected vs. what happened" — because
that's what gets good answers.
```

Pattern 8: Explain The Tradeoff

Use when: someone told you to do something a certain way and you don't understand the *cost* — or you keep seeing two idioms and want to know which is "right."

```
Help me understand a tradeoff I keep running into in {{language/domain}}.

I keep seeing {{idiom A}} and {{idiom B}} used for the same job. For example:
{{short code examples of each}}

Please:
1. Explain the real difference between them in plain language.
2. When you'd pick A over B, and when you'd pick B over A.
3. What a junior might do wrong with each.
4. How the choice changes as a codebase grows or as the team gets bigger.
5. One rule of thumb I can remember.

Give me a decision I can carry with me, not a lecture.
```

Pattern 9: "What Would a Senior Notice?"

Use when: you've written something that works, but you suspect it's not *good* — and you don't know what you don't know.

```
I wrote this code and it works, but I feel like a senior dev would spot things I'm missing. Show
me.

{{paste your code}}

Please:
1. List the things a senior developer would probably flag. For each, explain WHY in one sentence.
2. Sort them by how much they actually matter in production.
3. Tell me which of these I should fix now vs. which are "good enough for a junior to learn from
later."
4. Give me the single most important habit this code is missing.

Don't rewrite the code. I want to learn to see these things myself.
```

Pattern 10: The Interview Explainer

Use when: you're preparing for interviews or just want to be able to talk about your code.

```
I need to explain my project in an interview. Help me practice.

Here's my project: {{paste a description, a key file, or a feature}}

Please:
1. Give me a 60-second explanation of the project in plain language.
2. Then a 30-second "highlight" — the part I should lead with that shows the most skill.
3. Predict the 3 most likely follow-up questions a senior interviewer will ask, with model answers.
4. Point out the weakest part of my project that an interviewer might probe, and how to answer
honestly without underselling myself.
5. Give me one "resume line" describing this project.

Run me through it like a mock interview: ask me a question, wait for my answer, then give feedback.
```

The Prompt Library — 60+ prompts

Below are the core prompts, grouped into categories. Each has a name, the full copy-pasteable template, and a one-line "How to use." Fill in every `{{placeholder}}`.

Debugging & Error Diagnosis

Prompt 1. Parse and explain this error

```
I'm getting this error. Translate it, find the cause, and point me at the fix — but teach me along
the way.

Error: {{paste full error message + stack trace}}
Code: {{paste the relevant function}}

1. Explain the error in one plain sentence.
2. Identify the exact line most likely responsible; quote it.
3. Give the top 3 possible causes by probability.
4. Show the minimal fix for the most likely cause (small diff, not a rewrite).
5. Tell me what ONE thing to log to confirm the cause before I commit the fix.
```

How to use: Paste the real verbatim error. Do not paraphrase the error yourself — the exact text matters.

Prompt 2. Stack trace walkthrough

```
Walk me through this stack trace top to bottom as if I'm a junior.

{{paste stack trace}}

For each frame: tell me what layer it's in (my code, a library, the runtime) and whether I should
look at it or skip it. Then tell me which frame is the REAL source of the bug — usually not the top
one. End with: what should I search for in my code to find the bug at the source?
```

How to use: Best for errors that appear deep in library code. The point is learning which frames matter.

Prompt 3. Null/undefined debugging

```
I'm getting a "Cannot read property '{{prop}}' of {{null|undefined}}" error. Here's the code around
it:

{{paste code}}

1. In plain language, what is undefined/null at this point?
2. Trace BACKWARD: which earlier line returned the undefined value? Show the likely chain.
3. Give me 3 ways to guard against it, and the tradeoff of each (e.g. optional chaining vs. default
value vs. early return).
4. Tell me which fix is most appropriate for MY code and why.
5. Show me how to write a quick check/test that would've caught this.
```

How to use: This is the single most common junior error. If you've seen it once, this prompt teaches you to see it coming.

Prompt 4. Logic bug, not crash

```
My code runs without errors but produces the wrong result. Here's the function:

{{paste function + what it's supposed to do + example input/output that's wrong}}

1. Read the code carefully and tell me where the logic diverges from the intent.
2. Show me a small test case (input -> expected -> actual) that exposes the bug.
3. Explain the root cause in plain English.
4. Give me the fix and the reasoning behind it.
5. Ask me one question about the intended behavior that I need to clarify — there may be a spec
ambiguity, not a code bug.
```

How to use: Include the expected vs. actual output. If you omit it, the AI can't tell what "wrong" means.

Prompt 5. Intermittent / flaky bug

```
I have a bug that only happens sometimes, and I can't reproduce it reliably. Help me make it
reproducible.

Symptom: {{describe when it happens and when it doesn't}}
Code involved: {{paste the function or feature}}
What I know: {{what you've observed}}

1. Give me a checklist of things to check that commonly cause intermittency (race conditions, stale
state, timing, env differences, caches).
2. Rank them by how likely they are for my situation.
3. Tell me exactly how to add logging to catch the bad state when it occurs.
4. Suggest how to write a stress test or reproduction script that increases the odds of catching
it.
5. Walk me through the process of bisecting: narrowing down which feature or change introduced it.
```

How to use: Flaky bugs reward discipline, not guessing. Use this prompt the moment you're tempted to "just add a retry."

Prompt 6. Read a cryptic error message

```
The error doesn't look like normal English — it has codes, hex, or internal names. Explain it.

Error: {{paste the raw message}}
What I was doing: {{one line of context}}

1. What does this error message literally say, part by part? Break down any codes or identifiers.
2. What's the underlying problem in plain English?
3. What are the usual triggers for this specific error?
4. What's the fastest way to confirm it's what I think it is?
5. What should I Google to learn more — give me the EXACT search terms, not a vague topic.
```

How to use: Use when the error is from a framework or tool (e.g. `EADDRINUSE`, `ENOENT`, cryptic React/hook errors). The search-term tip alone is worth it.

Prompt 7. Debugging strategy for a feature

```
I'm debugging a feature that isn't working end-to-end. Help me make a plan before I touch code.

Feature: {{describe the feature}}
Behavior: {{what it does now}} vs. what it should do: {{what it should do}}

1. Give me a step-by-step plan to isolate the first broken link in the chain (input -> logic ->
output).
2. Tell me which single log statement to add first.
3. Tell me what NOT to change yet, so I don't confuse the debugging.
4. If the problem turns out to be in {{specific layer — e.g. frontend/backend}}, what's the next
diagnostic step?
5. After it's fixed, what regression test should I write?
```

How to use: Use this BEFORE you start changing code. It prevents the junior habit of tweaking random things.

Code Review & Quality

Prompt 8. Review a pull request (as the author)

```
I'm about to submit a PR. Review my changes with the eyes of a senior reviewer before I send it.

PR summary: {{what the change does}}
Diff / changed files: {{paste the diff or list the files and paste key hunks}}

1. Point out anything that could break in production or under edge cases.
2. Flag missing error handling, validation, or edge cases.
3. Tell me if the naming and structure are clear.
4. What would a reviewer ask about this that I haven't thought of?
5. Give me a checklist of 5 things to verify before I submit.
6. Is there anything that would be a "nitpick" I can just fix now to avoid rounds of back-and-
forth?
```

How to use: Do this before opening the PR. It's the cheapest code review you'll ever get.

Prompt 9. Review a PR (as the reviewer)

```
I need to review someone else's PR. Help me do a real, useful review — not just "LGTM."

PR summary: {{what it does}}
Diff: {{paste the diff}}

1. Give me 5 specific questions to ask the author about the changes.
2. Point out the most likely bug or edge case, if you see one.
3. Tell me what's good that I should explicitly compliment (reviews aren't only criticism).
4. Help me write the review comment in a professional, kind tone.
5. Flag anything that would block merge vs. could be addressed later.
```

How to use: Reviewing is a skill, not a duty. Use this to learn what to look at as you gain experience.

Prompt 10. Code smells detector

```
Scan this code for common code smells and explain each one.

{{paste code}}

For each smell you find:
1. Name it (e.g. long function, duplicated code, magic numbers, god object, premature optimization,
dead code).
2. Point to the specific lines.
3. Explain why it hurts maintainability in 1-2 sentences.
4. Give a concrete refactor suggestion (but don't apply it — show me the approach).

End with: the ONE smell that matters most here, and why.
```

How to use: Run this on your own code weekly. Learning to *see* smells is the fastest route to writing senior-level code.

Prompt 11. Is this code "good"?

```
I wrote this and I don't trust my own taste. Evaluate it honestly.

{{paste code}}

Grade it against these dimensions (A-F, with a one-line justification each):
- Correctness
- Readability
- Testability
- Performance (does it matter here?)
- Adherence to {{language}} idioms

Then: tell me the single highest-leverage change to improve it, and one thing that's genuinely
fine.
```

How to use: Use after you've written something that works. This calibrates your internal critic.

Prompt 12. Security quick-scan

```
Review this code for security issues — I'm a junior and I want to learn, not just be told.

{{paste code}}

1. Flag any: injection risks, unvalidated input, hardcoded secrets, insecure defaults, privilege
issues, or unsafe data handling.
2. For each, explain the exploit in plain terms and the fix.
3. Rank by severity.
4. Give me a "mental checklist" of security questions to ask about any code I write in the future.
5. Tell me which things I should NOT try to fix myself (e.g. auth protocols) vs. what's a safe fix
for my level.
```

How to use: Use on any code that touches user input, auth, or data. Keep the mental checklist.

Prompt 13. Readability pass

```
Make this code more readable WITHOUT changing its behavior. Teach me the changes.

{{paste code}}

1. Show me the refactored version with clear naming, small functions, and good structure.
2. Then give me a BEFORE/AFTER explanation of each change and WHY it's more readable.
3. Flag anything that's a matter of taste vs. objectively better.
4. If the original was fine, say so — don't refactor for its own sake.
```

How to use: The value is in the after-explanation. If you skip it, you just got rewritten code you don't understand.

Prompt 14. Review against team standards

```
I need my code to match {{team/library style guide}}. Check it.

Code: {{paste code}}
Style guide: {{the rules — e.g. ESLint/Airbnb/Prettier config, or naming conventions}}

1. List every deviation, with the exact location.
2. Tell me which are enforced by the linter (auto-fixable) vs. convention-based (need my judgment).
3. Highlight anything where MY code is actually better than the guide, so I know not to blindly
change it.
4. Give me the one-liner to run the auto-fixer for my stack (e.g. `npx eslint --fix`).
```

How to use: Use when a teammate's review is purely cosmetic. It helps you separate "must fix" from "taste."

Prompt 15. "Is this the idiomatic way?"

```
I wrote this in {{language}} but I'm not sure it's idiomatic. Show me what a native developer would
write.

{{paste code}}

1. Point out anything that looks "translated" from another language (e.g. for-loops where map/
filter fits, manual string building where template literals fit).
2. Show the idiomatic version and explain why it's preferred.
3. Tell me when the non-idiomatic version is actually fine.
4. Give me 2 idioms in {{language}} I probably don't know yet that would improve this code.
```

How to use: Great for language switchers (e.g. Java -> Python, Python -> TS). Kills the "translated code" tell that seniors notice immediately.

Learning New Concepts

Prompt 16. Learn a concept end-to-end

```
Teach me {{concept}} from zero to practical. I have {{X}} months of experience in {{language}}.

Structure your answer as:
1. What is it? (plain language + analogy)
2. What problem does it solve? What would I do without it?
3. A minimal working example in {{language}} that I can run.
4. A real-world scenario where I'd actually reach for it.
5. The 3 most common misconceptions juniors have.
6. Three practice exercises, from easy to hard, with hints (not full solutions).
7. A way to verify I've understood it.

Don't compress this. I want to actually learn it, not get a summary. If it's too big for one
answer, give me day 1 and ask if I want day 2.
```

How to use: The killer line is #6 — actual exercises. Do them. Don't skip to #4.

Prompt 17. Fill a knowledge gap

```
I'm following a tutorial on {{topic}} and I got lost at {{point}}. Help me fill the gap.

What I understood up to that point: {{explain}}
What confuses me: {{your confusion}}
The code/step that tripped me: {{paste it}}

1. Re-explain just the confusing step from the ground up.
2. Tell me what prerequisite knowledge I might be missing that's causing the confusion.
3. Check my mental model: here's how I think it works — {{your model}} — tell me where I'm wrong.
4. Give me a simpler, smaller version of the same concept I can master first.
```

How to use: This is the tutorial-rescue prompt. It works because it forces you to articulate your partial understanding.

Prompt 18. "Why is it done this way?"

```
I keep seeing {{pattern/idiom}} in {{language}}, and I want to understand the WHY before I copy it.

Pattern: {{describe or paste an example}}

1. Why does this pattern exist? What historical or practical problem does it solve?
2. What would happen if I did the "obvious" simpler thing instead?
3. What are the costs of the pattern (complexity, performance, readability)?
4. When should I use it vs. skip it?
5. Give me one tiny example of a bug caused by NOT understanding this pattern.
```

How to use: Copying idioms you don't understand is how "works on my machine" bugs get born. This prompt prevents that.

Prompt 19. Teach me a library

```
I need to learn the library {{library}} to do {{goal}}. Teach me the 20%.

1. What are the 3-4 core concepts I MUST understand first? Explain each with an analogy.
2. Give me a minimal working example for my specific goal: {{goal}}.
3. What are the library's biggest footguns (ways juniors misuse it)? List the top 3 with examples.
4. What does the library do badly, and what should I use instead for those cases?
5. Give me the 2 most useful functions/methods for my goal, and when to reach for them.
```

How to use: The "20%" framing stops the AI from dumping the whole docs. You get the 80% of value you need.

Prompt 20. Learn by fixing

```
I want to learn {{topic}} by fixing a broken example. Create the exercise for me.

Topic: {{topic}}
Level: {{experience level}}
Language: {{language}}

1. Write a small, realistic program in {{language}} that uses {{topic}} but contains 3 deliberate
bugs or bad practices.
2. Don't tell me what they are. Give me the code and ask me to find and fix them.
3. After I tell you my fixes, review them and tell me what I caught and what I missed.
4. If I miss something, walk me through finding it step by step — don't just reveal it.
5. Then give me a harder version.
```

How to use: This is spaced repetition in disguise. The "find the bug" mechanic makes the knowledge stick far better than reading.

Prompt 21. Concept map

```
Help me build a mental map of {{topic/domain}}.

1. Give me the 5-7 core concepts, as a list with each concept's one-line definition.
2. Show me how they relate (which depend on which, which are alternatives to each other).
3. Mark which ones a junior MUST understand first vs. which are "later" concepts.
4. For each core concept, give me the one question that proves I understand it.
5. Tell me what to study next after this map.
```

How to use: Use when a topic feels like a wall of unrelated terms. This turns it into a structure you can navigate.

Prompt 22. Compare with something I know

```
I understand {{concept A}}. Teach me {{concept B}} by relating it to A.

For example: I know how to use an array. Teach me a hash map/object by contrasting it with arrays.

Please:
1. Explain B as "A, but different in these ways: ..."
2. Give a side-by-side code example showing A and B doing the same task in {{language}}.
3. Tell me when A is the right tool and when B is.
4. Give me a "gotcha" where someone using A's mental model for B gets bitten.
```

How to use: Analogies and contrasts are how human memory works. Anchoring new concepts to known ones makes them stick.

Prompt 23. A better way to do what I did

```
I just wrote this code to accomplish {{goal}}. I suspect there's a cleaner way. Teach me it.

My code: {{paste}}
My goal: {{goal}}

1. Identify the "shape" of what I'm doing (e.g. mapping, filtering, reducing, state management,
data fetching).
2. Tell me the idiom/built-in that's designed for exactly this shape.
3. Show the cleaner version side by side.
4. Explain why it's better — and honestly tell me if it's only marginally better (sometimes my
version is fine).
5. Give me the name of the pattern so I can recognize it next time.
```

How to use: This converts every "I wrote something clunky" moment into a lesson. Do it with code you're embarrassed about — that's where the growth is.

Refactoring & Code Quality

Prompt 24. Refactor for readability

```
Refactor this code to be more readable, but KEEP the behavior identical. Show your work.

{{paste code}}

For each change, tell me:
1. What you changed and why.
2. The "before" and "after" for the specific part.
3. Whether the change is objective or a matter of taste.

Rules: do not change behavior. Do not "improve" performance unless asked. Do not rename things
without explanation. If the code is already good, say so.
```

How to use: The "show your work" rule is what turns a rewrite into a lesson. Refuse answers that just dump a new file.

Prompt 25. Extract functions

```
This function is too long and does too much. Help me split it.

{{paste the function}}

1. Identify the distinct responsibilities inside it.
2. Propose a new structure: what functions to extract, with names and what each takes/returns.
3. Show the refactored version.
4. Explain what made it hard to read originally and how the new structure fixes that.
5. Tell me how the extracted functions could be unit-tested independently.
```

How to use: Long functions are the #1 junior smell. This teaches the decomposition habit.

Prompt 26. Remove duplication

```
I have duplicated code in these places. Unify it.

The duplicates: {{paste snippets or list files/lines}}

1. Identify the shared essence and the differences between the copies.
2. Propose the cleanest abstraction (a function, a base class, a shared util, a config value).
3. Show the refactored version.
4. Explain the tradeoff: when is extracting shared code a bad idea (over-abstraction)? Be honest.
5. Give me a rule of thumb for when to extract vs. leave it.
```

How to use: Duplication cleanup is safe, high-value practice. The honesty requirement (step 4) stops you from over-engineering.

Prompt 27. Rename for clarity

```
Review the naming in this code. Bad names are a real bug.

{{paste code}}

1. List every name that doesn't clearly describe what it holds/does (variables, functions, classes,
params).
2. For each, propose a better name and explain why.
3. Point out any names that are actively misleading — they say one thing but do another.
4. Give me the thought process for naming well. a rule I can use next time.
5. Show the renamed version, but only the names you'd change.
```

How to use: Naming is the skill juniors undervalue most. The "rule you can use" line is the real takeaway.

Prompt 28. Simplify conditionals

```
This conditional logic is hard to follow. Simplify it without changing behavior.

{{paste the conditionals/early returns}}

1. Point out nested ifs, inverted conditions, or long boolean chains.
2. Suggest early returns, guard clauses, or a lookup table where appropriate.
3. Show the simplified version.
4. Explain the "happy path first" principle if it applies.
5. Tell me if any of it could be pushed down into the data layer instead of the logic.
```

How to use: Nested conditionals are a classic junior tell. This teaches flattening and guard clauses.

Prompt 29. Refactor toward testability

```
This code is hard to test. Refactor it so it becomes testable.

{{paste code}}

1. Explain WHY it's hard to test (hidden dependencies, hardcoded values, global state, doing too
much).
2. Propose a refactor using dependency injection or separation of concerns.
3. Show the refactored version.
4. Show me the test you could now write that you couldn't before.
5. Explain the principle (e.g. "inject dependencies, don't create them") in plain terms.
```

How to use: This is the bridge between "it works" and "it's testable" — the biggest quality jump a junior can make.

Prompt 30. The "magic number" cleanup

```
My code has hardcoded values and magic strings scattered around. Clean it up.

{{paste code}}

1. Find every magic number, magic string, and hardcoded config value.
2. Propose where they should live (named constants, config file, environment variables, enum).
3. Show the refactored version.
4. Explain the difference between values that SHOULD be constants vs. values that are fine inline.
5. For anything sensitive (API keys, URLs, credentials), tell me the right way to handle it — and
warn me if one is already committed to a repo.
```

How to use: Magic values are readability debt. This also sneaks in security awareness for secrets.

Prompt 31. The "shotgun surgery" refactor

```
I have a change that requires editing many files, and it feels wrong. Help me understand and fix
the structure.

The change: {{describe what you keep having to edit}}
The files involved: {{list them}}

1. Diagnose the underlying design problem (e.g. a responsibility spread across many places, missing
abstraction, tight coupling).
2. Propose a refactor that would make this change a single, localized edit.
3. Sketch the new structure.
4. Warn me about the risk of doing this refactor now vs. later.
5. Give me a safe, incremental plan so the refactor doesn't break everything at once.
```

How to use: The "shots felt" feeling is an early warning of a design problem. Learn to listen to it.

Prompt 32. Dead code / cleanup

```
Help me find code that can be safely removed or is never used.

My codebase summary: {{describe the project, or paste a file}}
What I suspect: {{e.g. unused imports, unused functions, commented-out code, dead branches}}

1. Teach me the commands to detect dead code in my stack (e.g. `npx tsc --noEmit`, linters, `grep`
approach).
2. Tell me what I should NOT delete even if it looks unused (e.g. public API, things used by tests,
magic "side effect" code).
3. Give me a safe deletion checklist.
4. Tell me how to verify the app still works after removing something.
```

How to use: Cleanup is satisfying and safe — as long as you check for surprise dependencies. This prompt enforces that check.

Testing & TDD

Prompt 33. Write tests for my function

```
Write tests for this function. I'll review every one before I trust it.

{{paste the function}}

1. List the edge cases worth testing (empty input, null, extremes, types, error cases).
2. Write the tests in {{testing framework}} covering them.
3. For each test, tell me what behavior it locks in.
4. Tell me which edge case I'm probably not thinking of.
5. Do NOT write tests that just mirror the implementation — I want behavioral tests.
```

How to use: The "review every one" stance matters — a test that just mirrors code is worthless.

Prompt 34. Arrange-Act-Assert help

```
Help me write cleaner tests. I'm learning the Arrange-Act-Assert structure.

My current test: {{paste}}

1. Identify the Arrange / Act / Assert sections in my test.
2. Point out where it mixes them or skips one.
3. Rewrite it with clear sections and good test names.
4. Teach me the 3 rules of a good unit test name.
5. Give me a short list of other test smells to avoid (testing implementation details, asserting
too much, etc.).
```

How to use: Test structure is grammar. Learn it once and every test you write improves.

Prompt 35. What should I test here?

```
I wrote this feature and I'm not sure what to test. Help me decide.

Code: {{paste the feature}}

1. List the behaviors worth testing, ranked by how much a bug there would hurt.
2. For each, say whether it's a unit, integration, or end-to-end test.
3. Tell me what NOT to test (implementation details, third-party libraries, things that change
constantly).
4. Propose the minimum set of tests that give real confidence.
5. Explain the testing pyramid briefly, and where this feature sits.
```

How to use: "What to test" is the hardest judgment call. This prompt builds that judgment.

Prompt 36. Mock vs. real dependency

```
I need to test code that depends on {{dependency}}. Should I mock it or use it real? Help me decide
and implement.

Dependency: {{the thing — API, DB, time, random, etc.}}

1. Explain the tradeoff: mock (fast, isolated, brittle) vs. real (slow, flaky, realistic).
2. Tell me what the best practice is for THIS kind of dependency.
3. Show me how to mock/fake it in {{testing framework}} the idiomatic way.
4. Show me one test using the fake and one using the real thing.
5. Teach me the rule: "mock at the boundary" — what that means here.
```

How to use: Mocking wrong is a common junior failure (over-mocking). This gives you the judgment.

Prompt 37. Test-driven development — walk me through it

```
I want to practice TDD properly. Coach me through adding a new feature test-first.

Feature: {{describe the feature, e.g. "a function that returns the cheapest item in a cart"}}

Walk me through:
1. Write the FIRST failing test — the simplest behavior that proves the feature's core works. Show
me it.
2. Let me try to make it pass. I'll paste my attempt. Then coach me.
3. Once it passes, tell me the next test to add that forces more behavior.
4. Keep going until the feature is complete, test by test.
5. At the end, tell me what refactoring to do (the third step of red-green-refactor).

Important: one tiny step at a time. Don't dump the whole feature at once. Ask me for my code after
each step.
```

How to use: This is the single best way to learn TDD — by doing it under coaching. Commit to the one-step-at-a-time rule.

Prompt 38. My test is flaky

```
My test passes sometimes and fails other times. Diagnose it.

Test: {{paste the test}}
What I've noticed: {{describe the flakiness pattern}}

1. List the common causes of flaky tests (timing, shared state, order dependence, network,
randomness, un-cleaned data).
2. Point to which is most likely given my symptoms.
3. Show me how to make it deterministic.
4. Teach me how to run tests in random order/parallel to surface hidden dependencies.
5. Give me a rule to prevent flaky tests from the start.
```

How to use: Flaky tests destroy trust in the suite. Fixing them early protects your team's confidence.

Prompt 39. Test coverage — is it enough?

```
Here is my test coverage report. Tell me if the important stuff is actually covered.

Report: {{paste coverage output}}
Key files: {{list the most important ones}}

1. Point out the files with high coverage that are still risky (coverage ≠ good tests).
2. Find important logic with low/no coverage that I should add tests for.
3. Tell me the "line coverage is a trap" caveat and how to evaluate coverage meaningfully.
4. Suggest the 3 tests that would add the most real safety.
5. Give me a realistic coverage target and where to focus next.
```

How to use: Use after running `coverage` / `nyc` / `jest --coverage`. This teaches reading coverage critically, not worshiping the number.

Prompt 40. Test my refactor didn't break anything

```
I just refactored {{file/function}}. Verify the behavior is unchanged.

Original code: {{paste the before version}}
New code: {{paste the after version}}

1. Read both and identify any behavioral differences (not just style).
2. If there are differences, flag them loudly — this is a bug or an intentional change, tell me
which.
3. Propose the key tests that would catch behavior drift between the two versions.
4. Tell me what to manually check if I don't have those tests yet.
```

How to use: Refactors + a safety net = confidence. This catches the "I renamed something and broke a subtle edge case" class of bug.

Architecture & Design

Prompt 41. Design a small feature

```
Help me design a {{feature/component}} before I write code.

Requirements: {{describe what it must do}}
Constraints: {{tech stack, team size, time}}
Where it fits: {{the surrounding system}}

Walk me through the design as a mentor:
1. Ask me 3 clarifying questions first. Wait for my answers.
2. Propose an architecture: the modules/components and how they talk to each other.
3. Draw it as a text diagram.
4. Identify the data flow.
5. Point out the design decisions that are hard to reverse, so I think twice.
6. List what I should prototype FIRST to retire the biggest risk.
```

How to use: Design before code is what separates "feature slinger" from engineer. The clarifying questions step is the most important line.

Prompt 42. Pick an architecture

```
I'm deciding between these architectures/approaches for {{project}}:

Option A: {{describe}}
Option B: {{describe}}

My project: {{describe scale, team, timeline, future plans}}

1. Compare them on: complexity, development speed now, maintenance later, testing difficulty,
hiring/team familiarity.
2. Recommend one for MY situation, with reasoning.
3. Tell me the "slippery slope" of each — the bad thing that tends to happen if the team takes the
wrong turn.
4. Suggest a pragmatic middle ground if one exists.
5. What's the minimum viable version I can build to validate the choice before committing?
```

How to use: Architecture choices are tradeoffs, not absolutes. This forces the tradeoff table into the light.

Prompt 43. "What does 'separation of concerns' mean HERE?"

```
Apply separation of concerns to this code and teach me the principle.

{{paste code}}

1. Identify the distinct concerns currently tangled together (e.g. UI, business logic, data access,
formatting).
2. Propose how to split them into layers/modules.
3. Show the restructured version.
4. Explain the benefit and the cost (more files, more indirection).
5. Give me the test that gets easier after the split.
```

How to use: This is the core of good architecture. Master it and most other architecture skills follow.

Prompt 44. Design for change

```
This code is hard to change when requirements evolve. Help me make it more adaptable.

Code: {{paste}}
A likely future change: {{describe what's coming, e.g. "add a new payment provider"}}

1. Identify what makes it rigid (hardcoded branches, direct coupling to a specific provider, etc.).
2. Show me the pattern that would make the future change a small, isolated edit (e.g. strategy
pattern, interface, config).
3. Implement the refactor.
4. Warn me: when is it premature to design for change? (YAGNI)
5. Give me the judgment rule for when to prepare for change vs. wait until it's needed.
```

How to use: Balance is everything. This teaches both the pattern and the discipline not to over-build.

Prompt 45. Data modeling help

```
I'm designing the data model for {{feature}}. Help me get it right.

Entities involved: {{what things exist and how they relate}}
What the app does with them: {{the operations}}

1. Propose the core entities, their key fields, and the relationships.
2. Point out design questions I'm missing (e.g. does an order capture a snapshot of price at
purchase time?).
3. Warn me about common modeling mistakes (over- or under-normalization, storing derived data,
etc.).
4. Give me a sample set of records that would exercise the model.
5. Tell me which decisions are expensive to change later.
```

How to use: Data modeling mistakes are costly. This surfaces the questions you don't know to ask.

Prompt 46. API design

```
Help me design the API for {{feature}}.

Purpose: {{what consumers need}}
Consumers: {{frontend? third party? internal services?}}

1. Propose the endpoints (or functions) with methods, paths, request and response shapes.
2. Explain the REST (or your chosen style) conventions you're following.
3. Show me example request/response JSON.
4. Tell me what error cases to handle and how to represent them.
5. Flag anything that would be painful to change later (e.g. returning the wrong shape, naming).
6. Give me a rule for when to use path param vs. query param vs. body.
```

How to use: APIs are contracts. Getting shapes right early saves a lot of churn.

Prompt 47. Handle async/error architecture

```
I'm building {{feature}} and need to design how it handles async work and errors.

The operations: {{what can fail, what's slow, what's background}}
Current approach: {{describe or paste}}

1. Tell me the failure modes I haven't considered (timeouts, partial failures, retries,
idempotency).
2. Propose an error-handling strategy: what's retryable, what should fail fast, what needs a queue.
3. Show me the pattern for wrapping async operations with consistent error handling.
4. Explain how to make retries safe (idempotent operations).
5. Give me a checklist to audit my async code for these issues.
```

How to use: Async/error handling is where production weirdness lives. This builds the mental model before bugs find you.

Prompt 48. Review my architecture

```
Review the architecture of this feature/small system as a senior architect.

Structure: {{describe the modules and their dependencies, or paste a diagram}}
Data flow: {{describe or paste}}

1. Point out coupling or dependency issues between modules.
2. Flag anything that would be painful to test.
3. Tell me where a junior's instinct (like putting all logic in the component) created a design
debt.
4. Suggest one structural improvement, explained and justified.
5. Give me the "smells" to watch for in my own future designs.
```

How to use: Run this after you've wired up a feature. It calibrates your architecture radar.

Git, Code Review & Collaboration

Prompt 49. Write a good commit message

```
Help me write a commit message for my changes.

What I changed: {{describe the change}}
Why: {{the reason}}

1. Write a conventional commit message (type: short summary) following {{convention — e.g.
conventional commits}}.
2. Give me the body with the "what" and "why" — not the how.
3. Tell me if my change should be split into multiple commits (if it does too much).
4. Show me a bad version of the same message so I can see the difference.
```

How to use: A commit message is communication to your future self and your team. This makes it a habit.

Prompt 50. Fix a bad commit history

```
I made a mess of my git history. Help me fix it safely.

Current situation: {{describe — e.g. "I made 10 tiny commits", "I committed a secret file", "I need
to split a commit", "I need to rewrite author info"}}
What I want: {{describe the desired end state}}

1. Tell me the EXACT commands in order.
2. For each command, warn me about what could go wrong and how to bail out.
3. Tell me how to check if my local history has been pushed (safe to rewrite) or not (dangerous).
4. Give me the "oh no, I need to undo" escape hatch for each step.
5. Tell me if this is a situation where I should ask a senior teammate instead of DIY.
```

How to use: History rewriting is powerful and risky. This prompt keeps you safe by making consequences explicit.

Prompt 51. Resolve a merge conflict

```
I have a merge conflict and I don't fully understand the two versions. Help me resolve it.

The conflicted file: {{paste the conflict markers}}
The branch/file context: {{describe what each side was trying to do}}

1. Explain what each side of the conflict is doing, in plain language.
2. Tell me which side (if either) is "correct" or whether they need to be merged semantically.
3. Show me the correctly merged result.
4. Explain how to think about conflicts generally (they're not a failure; they mean two changes
touched the same lines).
5. Give me a strategy to minimize future conflicts.
```

How to use: Conflicts are a learning moment, not a crisis. Understanding both sides is the real skill.

Prompt 52. Rebase vs. merge vs. squash

```
Help me understand when to use rebase, merge, and squash-merge.

My situation: {{describe your branch workflow and what you're about to do}}

1. Explain each operation in plain language (what it does to the history).
2. Explain the tradeoffs: clean history vs. preserving context vs. collaboration safety.
3. Tell me what your team's workflow ({{e.g. GitHub flow, trunk-based, Git Flow}}) implies here.
4. Give me a decision rule I can apply.
5. Warn me about when NOT to rebase (shared branches!).
```

How to use: This is a perennial junior source of anxiety. Learn the mental model, not just the commands.

Prompt 53. Write a PR description

```
Help me write the pull request description for my branch.

My changes: {{describe what I did and why}}
Branch: {{branch name}} -> {{target branch}}
Anything special: {{e.g. "this changes the DB schema", "this fixes a bug", "needs testing"}}

1. Write a clear PR description with: summary, motivation, and test plan.
2. Point out anything I should flag to reviewers (risky areas, decisions I made alone).
3. Tell me what to include so a reviewer can review fast (screenshots if UI, how to reproduce).
4. Give me a template I can reuse for all my PRs.
```

How to use: A good PR description gets you faster, better reviews. This is a career skill.

Prompt 54. Respond to review comments

```
I got review comments on my PR. Help me respond professionally and fix them.

My code: {{paste the relevant code}}
The review comments: {{paste them, verbatim}}

1. For each comment: explain what the reviewer is really asking for.
2. Tell me which comments are blocking vs. nits vs. informational.
3. Help me write a respectful, concise reply for each.
4. Show me the code change each comment likely requires.
5. If a comment is wrong or based on a misunderstanding, help me respond without being defensive.
```

How to use: Handling feedback gracefully is a soft skill that tech reviews rarely teach. This gives you the words.

Prompt 55. Onboard into a codebase

```
Help me understand this codebase quickly. I'm new to it.

Project: {{describe or paste the README}}
What I need to do: {{your first task}}

1. Give me a map: the main folders, what each is for, and where the "core" lives.
2. Tell me where I should start reading (the entry point, the main data flow).
3. Tell me how to run it and run the tests.
4. Identify the most important file I should understand first for my task.
5. Tell me what conventions to follow (naming, patterns, test style).
6. Warn me about the codebase's quirks (known tricky areas).
```

How to use: Read the README first, then run this. It dramatically shortens ramp-up time.

Prompt 56. Prepare a code walkthrough

```
I need to walk my team through my code. Help me prepare.

My changes: {{paste the key code or describe the feature}}

1. Give me a 3-minute walkthrough script: what to show, what to say, in order.
2. Point out the parts I should draw attention to (the clever/important bits) and parts to skip.
3. Anticipate the 3 questions people will most likely ask, with answers.
4. Help me present the design decisions as choices I made, not just "I did X."
5. Give me a closing line that invites good feedback.
```

How to use: Walkthroughs build your reputation and your understanding. The "design decisions as choices" tip is gold.

Performance & Debugging

Prompt 57. My code is slow — profile it

```
My {{feature/function}} is slow. Help me find where the time goes.

The code: {{paste the hot path}}
Symptom: {{e.g. "spins for 10s on this input"}}

1. Tell me how to profile this in my stack (the exact tool: e.g. perf, cProfile, Chrome DevTools,
React Profiler), with the commands.
2. Given the code, list the likely bottlenecks ranked by probability (N+1 queries, O(n²) loops, re-
renders, blocking I/O).
3. Show me where to add timing logs to confirm before optimizing.
4. ONLY after confirming: show the optimization, and explain the before/after complexity.
5. Tell me what NOT to optimize (premature optimization warning).
```

How to use: Measure first, optimize second. The prompt bakes that order in.

Prompt 58. N+1 query problem

```
I suspect my data layer has the N+1 problem. Diagnose and fix it.

Code: {{paste the query/loop pattern}}
What I'm seeing: {{slow page loads, many identical queries}}

1. Confirm whether it's N+1 and explain what that is in plain terms.
2. Show me where the repeated query happens.
3. Fix it using the idiomatic approach for {{ORM/library}} (eager loading, a JOIN, batching).
4. Show me the before/after and why it's faster.
5. Teach me how to spot N+1 in code review from now on.
```

How to use: N+1 is everywhere and juniors can't see it. This trains the eye.

Prompt 59. React re-render / perf

```
My {{React/Vue/Angular}} app re-renders too much and is janky. Help me fix it.

Component: {{paste the component}}
Symptom: {{describe the jank/lag}}

1. Explain WHY it's re-rendering (state placement, prop identity changing every render, context).
2. Tell me how to confirm with the devtools (React Profiler etc.).
3. Show the targeted fix (memoization, memo, splitting state, lifting/lowering state).
4. Explain the cost and pitfalls of over-memoizing.
5. Give me the rule for when memoization is worth it.
```

How to use: Performance in the UI is about reducing unnecessary work. The "what not to do" warning keeps you from cargo-cult `useMemo`.

Prompt 60. Optimize an algorithm

```
This algorithm is too slow. Help me pick a better one and implement it.

The problem: {{describe what it computes}}
Current approach: {{paste the code}}
Input sizes: {{e.g. "lists up to 100k items"}}

1. Analyze the current time/space complexity and explain what that means in plain terms.
2. Propose the more efficient approach, with its complexity.
3. Show the implementation.
4. Explain the tradeoff (often memory for speed).
5. Tell me if the optimization is worth it at MY input sizes — sometimes the "slow" version is
fine.
```

How to use: Complexity analysis is the core computer science skill. "Is it worth it" keeps it grounded in reality.

Prompt 61. Memory / resource leak

```
My app's memory usage grows over time. Help me find the leak.

Symptom: {{describe — e.g. "RAM climbs until it crashes"}}
Suspected areas: {{e.g. event listeners, caches, timers, closures}}
Stack: {{language/framework}}

1. Explain the common leak patterns in my stack (unremoved listeners, growing caches, unclosed
connections, references held in closures).
2. Tell me how to inspect memory (heap snapshots, devtools, `process.memoryUsage`).
3. Walk me through a methodical hunt: which suspects to check first.
4. Show me the fix for the most likely pattern.
5. Give me habits that prevent leaks from the start.
```

How to use: Leaks are subtle. The methodical approach prevents rabbit holes.

Prompt 62. Slow startup / cold start

```
My app takes forever to start. Help me reduce startup time.

Symptom: {{describe the startup time and what you've measured}}
Stack: {{framework, build tool}}

1. List the common startup costs (large bundle, eager DB connections, heavy imports, sync file
reads).
2. Tell me how to measure what's actually slow.
3. Show me the idiomatic fixes (lazy loading, deferred init, code splitting).
4. Explain the tradeoffs (complexity, first-call latency vs. startup).
5. Give me a target and a way to verify improvement.
```

How to use: Measure first. The prompt insists on it.

Prompt 63. Benchmark two approaches

```
I need to decide between two implementations by measuring them fairly. Set up the benchmark.

Approach A: {{paste}}
Approach B: {{paste}}
The task: {{what they do}}

1. Write a benchmark harness in {{language}} (with warmup, enough iterations, and variance
reporting).
2. Explain the pitfalls to avoid (compiler optimizations, too-small inputs, not warming up,
measuring I/O).
3. Run it and report which is faster and by how much.
4. Tell me whether the difference matters at real-world scale.
5. Tell me which approach is better for OTHER reasons (readability, maintainability) even if it's
slightly slower.
```

How to use: Benchmarking well is rare even among seniors. The pitfalls section is the real lesson.

Documentation & Code Comments

Prompt 64. Write a README

```
Help me write a README for {{project}}.

Project: {{describe what it does}}
Audience: {{who will read it — teammates? future maintainers? users?}}
Tech stack: {{list}}

Structure it as:
1. One-line description
2. Quick start (install + run + test)
3. Project structure overview
4. Configuration / environment variables
5. Common tasks / scripts
6. Troubleshooting section
Write it in plain, friendly English. Don't assume prior knowledge, but don't be condescending.
Include the actual commands for my stack.
```

How to use: A README is the front door to your repo. Future-you will thank you.

Prompt 65. Document my functions

```
Write doc comments for these functions. I care about WHY more than WHAT.

{{paste the functions}}

For each function:
1. One sentence on what it does.
2. Parameters: what each is, what it expects, edge cases.
3. Return value and its type.
4. What it throws / error cases.
5. A usage example.
6. Any gotchas (side effects, state it modifies, performance implications).

Also: tell me which functions DON'T need docs (self-explanatory ones) so I don't over-comment.
```

How to use: The "which ones don't need docs" part is the lesson — good code needs fewer comments.

Prompt 66. Comment the *why*, not the what

```
This code has comments that just restate the code. Replace them with useful comments.

{{paste code}}

1. Identify comments that add no information (redundant "what" comments).
2. For each, tell me whether to delete it or replace it with a "why" comment.
3. Write the replacement comments explaining the *why* (the non-obvious reasoning, the constraint,
the workaround).
4. Point out places that NEED a comment but don't have one.
5. Give me the rule: comment the why, name the what, and let the code speak for itself.
```

How to use: This is the classic "comments lie" lesson. It teaches you to treat code as the primary documentation.

Prompt 67. Write changelog / release notes

```
Help me write a changelog entry for this release.

Changes: {{list what changed — features, fixes, breaking changes}}
Version: {{version number, if known}}

1. Write the entry in keep-a-changelog style (or the style my project uses).
2. Separate Added / Changed / Fixed / Removed / Breaking.
3. Flag anything that's a breaking change and write the migration note.
4. Tell me what to leave out (internal details, chores).
5. Review my existing changelog format first if I provide it.
```

How to use: Release notes are user-facing. This keeps them honest and useful.

Prompt 68. Document a tricky decision (ADR)

```
Help me write an Architecture Decision Record (ADR) for a decision I made.

The decision: {{what I chose}}
Alternatives: {{what I rejected and why}}
Context: {{the constraints and situation}}

Write it in this structure:
1. Title and status (proposed/accepted/superseded)
2. Context — the problem and constraints
3. Decision — what we chose, in one clear paragraph
4. Consequences — positive and negative, including what we're accepting
5. Alternatives considered — with brief reasoning for each rejection

Keep it concise and factual. An ADR is for future maintainers asking "why on earth did they do
this?"
```

How to use: ADRs are how teams preserve decision memory. Writing one is a senior-skill flex.

Career & Codebase Understanding

Prompt 69. Understand a legacy codebase

```
Help me understand this legacy codebase without getting overwhelmed.

Project: {{paste README or describe}}
My task: {{the feature/bug I need to touch}}

1. Which folders/files are the "living" parts vs. dead/legacy parts?
2. What is the single most important flow I must understand first?
3. Where are the biggest dependencies and the riskiest code to touch?
4. How do I run it, test it, and debug it?
5. What are the unofficial conventions (the way things are actually done here)?
6. Give me a roadmap: what to read in what order this week.
```

How to use: Legacy code is where juniors fear to tread. A map turns terror into a plan.

Prompt 70. Read a large file quickly

```
I have to understand a big file. Help me extract its essence fast.

File: {{paste it, or the first N lines + outline}}
How big: {{lines/scope}}
Why I need it: {{task}}

1. Give me a 5-line overview of what the file does.
2. List the main functions/classes and one-line responsibilities for each.
3. Which parts matter for MY task, and which can I safely ignore?
4. What's the data flow through the file?
5. What are the "gotcha" parts (global state, side effects, tricky coupling)?
```

How to use: Efficient reading of unfamiliar code is a superpower. This gives you the first pass.

Prompt 71. Estimate a task

```
Help me estimate how long a task will take, and break it down.

Task: {{describe}}
My experience with the stack: {{describe}}
Related code: {{describe or paste what I'll be touching}}

1. Break it into sub-tasks with rough time estimates each.
2. Tell me the risky/unknown parts that could balloon the estimate, and how to de-risk them first.
3. Give me a "spike" plan to research the unknowns before committing to the full estimate.
4. Tell me what to do if reality diverges from the estimate (when to escalate).
5. Give me the honest estimate range, not a single number.
```

How to use: Estimation is a career skill. The "spike first" advice is the pro move.

Prompt 72. Prepare for a code interview / take-home

```
Help me prepare for a {{type — take-home / technical interview / live coding}} on {{topic}}.

What I know: {{describe your current comfort level}}
The format: {{describe what I'm told to expect}}

1. Give me a study plan for the next {{timeframe}}.
2. List the core concepts/problems most likely to appear, and for each: the pattern that solves it.
3. Give me one representative practice problem with hints (not the full solution) to work through.
4. Tell me the common mistakes candidates make and how to avoid them.
5. For take-homes: tell me how to structure the submission (README, how to run, what to focus on).
```

How to use: Adapt the plan to your timeframe. The take-home structure advice (step 5) is often the difference between passing and not.

Prompt 73. Career check-in on my code

```
In 6 months I want to look back at this code and be proud of it. What should I do differently now?

Code: {{paste something you wrote recently}}
Where I am: {{describe your level and goals}}

1. What's the ONE habit that would improve my code the most over 6 months?
2. What patterns in this code will I cringe at in 6 months?
3. What should I learn next that most directly lifts the quality of what I write?
4. What should I NOT worry about right now (things that come with time)?
5. Give me a concrete weekly practice to build the habit.
```

How to use: This is your long-game prompt. Re-read it monthly.

Prompt 74. Understand my team's workflow

```
Help me understand how my team works so I can be a better collaborator.

Team context: {{describe — e.g. "we use Jira, do sprints, review PRs"}}
What I'm unsure about: {{describe your questions}}

1. Explain the typical lifecycle of a task in this setup, from ticket to deploy.
2. Tell me what to do before writing code (checking the ticket, understanding acceptance criteria,
asking clarifying questions).
3. Tell me what to do after (demo, update the ticket, pair with reviewer).
4. List the unwritten rules of a good team member at my level.
5. Give me a checklist for my first week on any new team.
```

How to use: The "unwritten rules" list is the part they don't teach you in bootcamp.

Prompt 75. "What did I learn from this bug?"

```
I just fixed a nasty bug. Help me turn it into a permanent lesson.

The bug: {{describe}}
The fix: {{what you changed}}
How I found it: {{describe your debugging journey}}

1. Write a postmortem in 5 bullet points (what happened, why, what I'd do differently, how to
prevent it).
2. Tell me what general principle this bug teaches (e.g. "always validate input", "don't trust
cached state").
3. Suggest a test or a check that would prevent this class of bug going forward.
4. Turn it into a one-line note I can remember.
5. Tell me if it's worth writing up for the team (or just for my future self).
```

How to use: Bugs are tuition. This prompt makes sure you graduate.

Debugging decision tree

When you hit a problem, don't guess. Find your symptom below, then jump to the prompt that matches. This table is deliberately small — it covers the ~12 situations that account for most junior debugging time.

Symptom	Likely cause	Use this prompt
`Cannot read property 'x' of undefined/null`	Data not loaded yet, or an object you expected is missing	**Prompt 3** (null/undefined)
`TypeError: x is not a function`	Wrong type; a `Promise` not awaited; wrong import	**Prompt 1** (parse error) + **Pattern 1** (async/await)
Correct code but wrong output	Logic bug or a spec ambiguity	**Prompt 4** (logic bug)
Error is cryptic / has codes or hex	Framework or tool-level error (port, `ENOENT`, hooks)	**Prompt 6** (cryptic error)
Stack trace is deep in library code	The real cause is my code a few frames down	**Prompt 2** (stack trace walkthrough)
Works on my machine, fails in prod	Environment difference (env vars, versions, OS)	**Prompt 5** (intermittent)
Happens only sometimes	Race condition, timing, stale state, shared state	**Prompt 5** (intermittent) + **Prompt 38** (if it's a test)
Page loads slowly / app lags	N+1 queries, wasted re-renders, blocking I/O	**Prompt 57** (profile) → **Prompt 58** (N+1)
Memory climbs until it crashes	Leak: listeners, caches, unclosed connections	**Prompt 61** (memory)
Tests pass sometimes, fail sometimes	Flaky test: timing, order dependence, shared state	**Prompt 38** (flaky test)
Feature works except one step	Broken link in the chain (input → logic → output)	**Prompt 7** (debugging plan)
App is slow to start	Huge bundle, eager init, heavy imports	**Prompt 62** (slow startup)
"It just doesn't work" with no error	The failure is silent — add a log at the boundary	**Prompt 7** + **Prompt 3**

How to use the tree: find your row, copy the linked prompt, fill in the placeholder with your real code/error. If you're between two rows, pick the one whose prompt is cheaper to run — a quick `console.log` beats an hour of theory.

Your first 30 days

You don't learn a skill by reading about it — you learn it by doing. Here is a week-by-week plan that walks you through the pack while you build or work on a real project. Swap the project in wherever it makes sense.

Week	Focus	What to do	Prompts / patterns to use
1	Foundation + read code	Pick one small feature of your project. Read it fully. Write a short "what it does" note. Run the existing tests. Write 3 new tests for the feature.	Pattern 4 (Code Explainer), Prompt 33 (write tests), Prompt 70 (read a big file)
1	Debug a real bug	Find a bug in your project (or add one deliberately in a scratch file). Debug it with the tree.	Prompt 3, 4, 7

Week	Focus	What to do	Prompts / patterns to use
2	Write better code	Rewrite that feature with better naming, smaller functions, and no magic numbers. Get a review via the pack.	Prompt 10 (smells), 24 (refactor), 27 (naming), 30 (magic numbers)
2	Learn one concept properly	Pick one concept you're shaky on. Use the ELI5J pattern, then do all three practice exercises.	Pattern 1 (Concept Explainer), Pattern 5 (Model My Thinking), Prompt 16
3	Refactoring + TDD	Take a second feature. First write failing tests (TDD style), then refactor the whole thing for readability and testability.	Prompt 37 (TDD coaching), 29 (testability), 40 (refactor check)
3	Architecture thinking	Draw the structure of your project (modules, data flow). Ask the architecture prompts.	Prompt 41, 43, 48
4	Ship it right	Open a real PR (or simulate one). Write the description, do a self-review, and respond to feedback.	Prompt 49, 53, 54, 8
4	Career + persistence	Do the long-game check-in and the postmortem on any bug you fixed this month. Set one habit for next month.	Prompt 73, 75

The habit rule: pick exactly one prompt you'll run every day for the next 30 days (most people choose **Pattern 3** — Review My Code — or **Prompt 32** — dead code). A daily 15-minute habit beats a six-hour weekend cram.

The "one thing" rule: every day, make one change to your code that is *smaller* than a whole feature. One renamed variable. One extracted function. One test. That's how senior habits are built — compounding, not rewriting.

FAQ

Q: Do I need to know about prompt engineering to use this?
A: No. Every prompt is written to be copy-pasteable — you only fill in the `{{placeholders}}`. The engineering (forcing explanations, demanding verification, refusing blind rewrites) is already baked in.

Q: Won't the AI just ignore the "teach me" instructions and dump code?
A: Sometimes it will. That's why each pattern ends with a verification demand. If an answer dumps code without explanation, reply with the follow-up: *"Explain the reasoning behind each change and tell me what to test to verify it."* The pack's prompts are written to survive that.

Q: Are these for a specific language or framework?
A: No. The prompts use `{{language}}` and `{{framework}}` placeholders and work for TypeScript, Python, Java, Go, React, Node, and anything else. The example explanations in the ELI5J section use TypeScript and Python because those are the most common junior stacks — but every template is language-agnostic.

Q: How is this different from just asking ChatGPT to fix my bug?
A: That gives you a fix you don't understand. This gives you a *lesson*. Every prompt is structured to make the AI explain, show alternatives, and ask you to verify — so the next time a similar problem appears, you solve it yourself. You also stay in control: nothing gets rewritten behind your back.

Q: I'm a complete beginner, not even a bootcamp grad. Is this for me?
A: The pack assumes you can read and write basic code (you know what a function, a variable, and a loop are). If you're pre-that, start with Pattern 1 and Pattern 19 until you're comfortable, then expand. The "first 30 days" plan assumes a project you can practice on.

Q: How should I actually use the prompts day-to-day?
A: Keep this file open in a tab. When you hit a wall, open the relevant section, copy a prompt, fill in the placeholders. Don't memorize prompts — memorize the *categories* (debug, review, learn, refactor, test, architect) so you know which section to open.

Q: Will the pack match my team's specific tools (Jira, GitHub, particular linters)?
A: The prompts take tool names as placeholders (`{{testing framework}}` , `{{convention}}`), so they adapt. When you run a prompt, just fill in your actual tool names and it will tailor the output to them.

Free lifetime updates — every future version is yours

This pack is a living document. As the AI landscape changes, as new frameworks rise, and as we learn what works (and what doesn't) with real junior developers, we update and expand these prompts.

When you buy, you get every future version — free, forever.

That includes:
- New prompt categories as the field evolves.
- Updated templates that reflect best-practice prompting.
- New worked examples in emerging languages and frameworks.
- Any bonus sections we add (more 30-day plans, more decision trees).

You'll be notified of each update and can download the latest version at no cost. No subscriptions, no upsells, no expiration.

The prompts you're holding are built to make you *less* dependent on AI over time — teaching you to reason, debug, and review like the engineer you're becoming. We keep improving the pack so it keeps up with you.

Now go build something. And when you're stuck — you know which section to open.

Junior Developer Code Prompt Pack — Learn and build faster with 60+ curated coding prompts that teach.